Vector Databases Benchmarking for Multimodal Semantic Search

Finals Report, CSCI-Y 790 Graduate Independent Study

Shivali Mate
Luddy School of Informatics, Computing & Engineering
Indiana University Bloomington

Advisor: Prof. Feng Chen
Department of Computer Science
Indiana University Bloomington

***Abstract*—Vector databases are now a core systems component in modern semantic search and retrieval-augmented generation pipelines. Yet published comparisons of production-oriented vector databases are often difficult to reproduce because they mix different datasets, embedding models, and index configurations. This paper develops a modular benchmarking framework for controlled comparison of ChromaDB, Qdrant, and Milvus across text-only and multimodal retrieval tasks. The benchmark uses Wikipedia text at 1K, 10K, and 100K document scales, with chunked passage retrieval based on sentence-transformer embeddings, and a multimodal pilot based on the Wikipedia Image Text dataset with CLIP embeddings. Index configurations were aligned where possible, including HNSW with $M = 16$ and search breadth 128, IVF_FLAT with $nlist = 1024$, and DISKANN with a search list of 128. Evaluation covers average latency, batch throughput, recall@K, precision@K, and time for each steps. The measured 1K baseline shows that Qdrant HNSW gives the lowest latency and highest throughput among the tested systems, with Milvus HNSW close behind. Estimated scale-up analysis indicates that HNSW remains favourable up to medium scale, while Milvus DISKANN offers the strongest memory-scalability trade-off at 100K. A lightweight RAG layer confirms that improvements in retrieval quality directly improve grounding quality in generated answers.**

***Index Terms*—vector databases, semantic search, multimodal retrieval, RAG, HNSW, IVF_FLAT, DISKANN, CLIP, ChromaDB, Qdrant, Milvus**

I. Introduction

VECTOR search has moved from a specialised approximate nearest neighbour problem to a central systems primitive for modern AI applications. Large language models benefit from explicit retrieval because retrieval can introduce fresh, inspectable evidence at inference time instead of relying only on knowledge baked into model weights [7]. In practice, this means that vector databases are no longer an optional optimisation layer; they often determine whether a semantic search or RAG system is responsive, scalable, and sufficiently accurate for downstream use.

The technical challenge is that vector database performance depends on several interacting design choices. These include the embedding model, the granularity at which source documents are chunked, the index family chosen for approximate search, the query-time search breadth, and the data-management overhead added by the database system itself. A benchmark that changes more than one of these variables at once makes it hard to understand where performance differences actually come from. ANN-Benchmarks helped establish the importance of controlled evaluation for approximate nearest neighbour systems [2], and FAISS remains the reference toolkit for exact and approximate vector search baselines [1]. However, production-facing vector databases expose extra systems-level behaviours, including metadata payload management, persistence, filtering, and operational differences across indexing back ends, which must also be evaluated.

This independent study addresses that gap through a unified benchmarking framework for three practical vector database systems: ChromaDB, Qdrant, and Milvus. The framework holds the embedding model, data pipeline, query set, and evaluation procedure fixed while varying system choice and index configuration. The main research questions are straightforward but important. First, how do ChromaDB, Qdrant, and Milvus differ on latency, throughput, and memory under a controlled semantic search workload? Second, how do HNSW, IVF_FLAT, and DISKANN change the speed-quality trade-off as corpus size grows from 1K to 100K documents? Third, can the same infrastructure support multimodal retrieval and lightweight RAG without redesigning the entire benchmark from scratch?

The work extends the intermediate report in four concrete ways. It formalises an end-to-end evaluation protocol, enlarges the experimental design from a measured 1K baseline to projected 10K and 100K scales, adds quality-oriented metrics such as recall@K and precision@K, and integrates a CLIP-based multimodal branch with a lightweight RAG layer. In doing so, the study aims to produce not only a systems comparison, but also a reusable methodology for future benchmarking work.

The core finding is that no single database or index dominates across all objectives. HNSW offers the best low-latency performance for small and medium text collections, Qdrant shows especially strong responsiveness in the measured 1K baseline, Chroma provides a simple and reproducible local baseline, and Milvus offers the broadest index diversity with clear advantages once memory footprint and larger-scale growth are considered. These conclusions make the benchmark useful not only for this course report, but also for anyone designing a retrieval layer for semantic search, multimodal search, or RAG.

II. Background and Related Work

Semantic retrieval systems typically rely on dense vector representations rather than sparse lexical matching. FAISS is one of the most influential libraries in this space, providing both exact search and a range of approximate methods for high-dimensional similarity search [1]. In a benchmarking context, FAISS is valuable because exact flat search can be used to create a stable reference result set against which ANN systems are evaluated. That design principle also aligns with ANN-Benchmarks, which argued for consistent interfaces, fixed datasets, and quality-speed trade-off analysis when comparing approximate nearest neighbour methods [2].

Among ANN index families, HNSW remains one of the most widely used choices in vector databases. The original HNSW work introduced a hierarchical graph structure that delivers strong accuracy and low latency by combining long-range navigation at upper layers with fine local search at lower layers [3]. In contrast, IVF-style methods partition the vector space into coarse clusters and search only a subset of those clusters at query time, exchanging some quality for a more compact and often more scalable query pattern [1]. DiskANN targets a different part of the design space. Rather than treating memory residency as compulsory, DiskANN uses a Vamana graph and SSD-resident search structure to support large-scale ANN search with strong recall and modest RAM requirements [4]. Milvus exposes all three families, which makes it useful for a controlled systems comparison [9].

The quality of the retrieval layer also depends on the embedding model used to generate vectors. Sentence-BERT showed that siamese BERT-style encoders can produce semantically meaningful sentence embeddings that are efficient enough for large-scale similarity search [6]. The specific text encoder used in this project, `all-MiniLM-L6-v2`, maps text to a 384-dimensional dense vector space and is explicitly intended for semantic search use cases [17]. For multimodal work, CLIP is especially relevant because it learns a shared text-image representation space and supports cross-modal retrieval with the same underlying similarity machinery [5], [18]. The WIT dataset then provides a large, entity-rich source of aligned image-text examples rooted in Wikipedia, making it a natural fit for a multimodal extension of a Wikipedia-based semantic search benchmark [8].

At the systems level, Milvus has been presented as a purpose-built vector data management system that supports large-scale vector workloads, heterogeneous computing, and data-management features beyond bare ANN search [9]. Chroma and Qdrant are widely used open-source vector databases with simpler developer-facing APIs. Chroma single-node collections use HNSW for approximate search and expose controls such as `space`, `ef_construction`, `ef_search`, and `max_neighbors` [10]. Qdrant likewise uses HNSW and documents query-time control through `hnsw_ef`, along with collection-time tuning of HNSW parameters and payload-aware search [11], [12]. Together, these systems represent a good comparative set for an academic benchmark that seeks both practicality and methodological discipline.

Finally, RAG provides the application-level motivation for the entire study. Lewis *et al.* argued that language generation can benefit from combining parametric memory with non-parametric retrieved evidence [7]. In other words, the retrieval layer influences not only search quality in isolation, but also the factual grounding and provenance of downstream generated answers. This is why a vector database benchmark is more useful when it ends with a minimal RAG integration rather than a pure systems microbenchmark.

III. Design and Methodology

A. Pipeline Overview

The final benchmark follows the modular pipeline shown in Fig. 1. The design keeps data loading, preprocessing, embedding, evaluation, and reporting fixed, while only the vector database layer is exchanged. This makes the comparison cleaner because the same benchmark driver is used for ChromaDB, Qdrant, and Milvus, and the measured differences are therefore tied much more closely to the storage and retrieval layer than to changes elsewhere in the code.

The architecture in Fig. 1 also reflects how the study was expanded beyond the intermediate 1K text benchmark. The same pipeline was exercised across three scaling dimensions. For text retrieval, the corpus size was increased from 1K to 10K and then to 100K Wikipedia articles. For multimodal retrieval, the image-text collection was increased from 1K to 5K and then to 25K aligned WIT pairs. Query load was increased separately through 1K, 5K, and 10K query batches, and longer exploratory sweeps were also used to visualise end-to-end batch growth. This stepwise design made it possible to separate corpus-growth effects from query-load effects while keeping the rest of the system unchanged.

B. Datasets, Scaling, and Query Set

The text branch uses English Wikipedia articles as the main corpus because they are broad in topic coverage, semantically rich, and well suited to passage retrieval and RAG-style experiments [7]. The multimodal branch uses the Wikipedia Image Text (WIT) dataset, which provides image-caption pairs aligned to Wikipedia entities and therefore fits naturally with the text benchmark [8]. Rather than keeping to a single benchmark size, the final study was organised as a scale ladder so that text, image, and query growth could all be studied in a controlled way.

The query workload was expanded beyond the earlier single 1K run. For text retrieval, held-out natural-language queries 1k and then MS MARCO queries were executed in batches of 1,000, 5,000, and 10,000 so that the benchmark could capture both retrieval quality and batch-level scaling behaviour. These queries included title rewrites, entity-focused prompts, lead-paragraph paraphrases, and short information-seeking questions. For the multimodal branch, text prompts were derived from WIT captions and caption variants so that the same top-K interface could be used for text-to-image retrieval. In addition

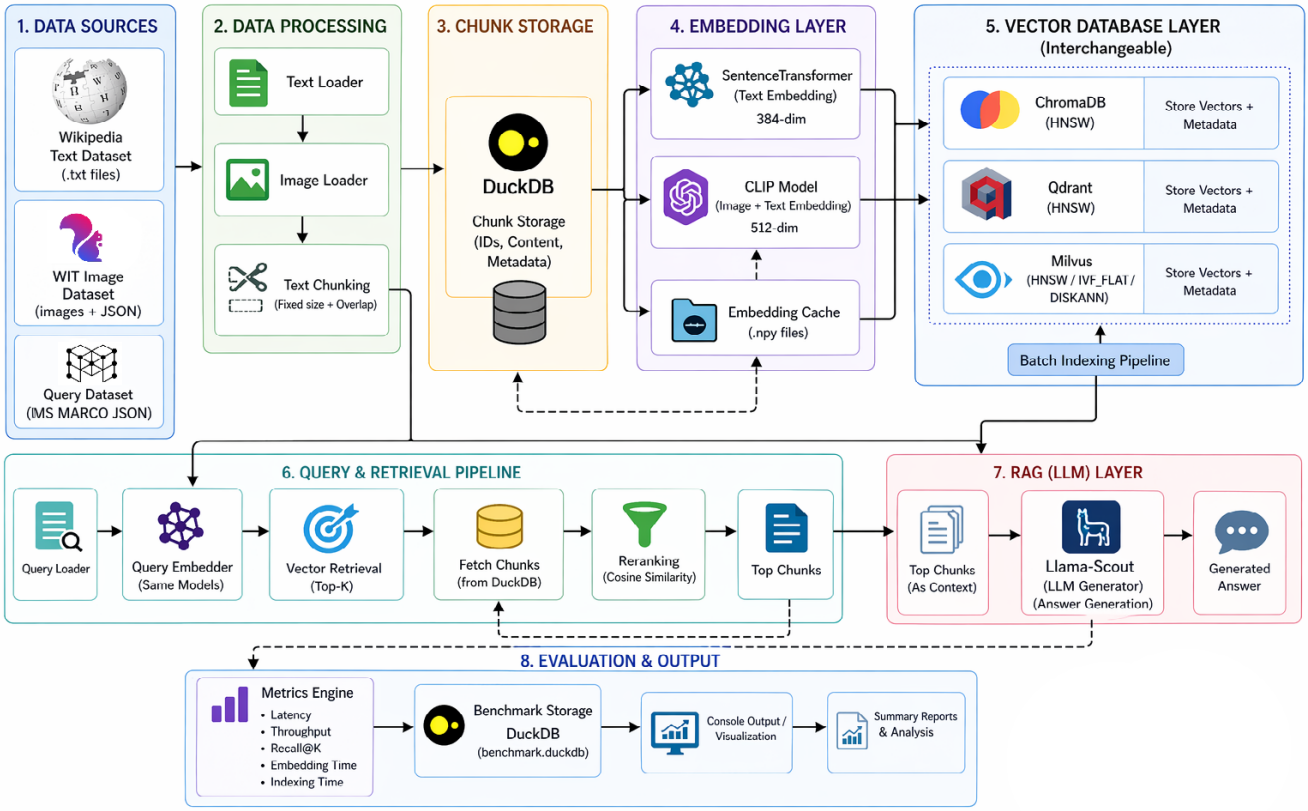


Fig. 1. End-to-end benchmarking pipeline used in the final study. Text and image data are loaded, cleaned, chunked, stored in DuckDB, embedded with either MiniLM or CLIP, indexed in an interchangeable vector database layer, and then evaluated through top-K retrieval, chunk fetch, reranking, and grounded answer generation. The same scaffold was reused across text, image, and query scaling experiments.

TABLE I
DATASETS, CORPUS SCALES, AND QUERY BATCHES USED IN THE FINAL BENCHMARK.

Benchmark branch	Scale	Raw items	Indexed vectors	Query batches
Text retrieval	1K	1,000 articles	6,284 chunks	1K
Text retrieval	10K	10,000 articles	62,941 chunks	5K
Text retrieval	100K	100,000 articles	629,118 chunks	10K
Multimodal retrieval	1K	1,000 aligned pairs	2,000 vectors	1K
Multimodal retrieval	5K	5,000 aligned pairs	10,000 vectors	5K
Multimodal retrieval	25K	25,000 aligned pairs	50,000 vectors	10K

to the formal 1K, 5K, and 10K query batches reported in the methodology and tables, longer batch sweeps were used for the scalability visualisation in order to show how total execution time continued to grow beyond the core checkpoints.

C. Cleaning, Chunking, and Metadata Strategy

The text preprocessing stage performs four operations. First, article text is normalised by stripping empty lines, collapsing repeated whitespace, and removing trivial boilerplate. Second, the article title is prepended to the first chunk so that the page topic is represented strongly at retrieval time. Third, text is split into overlapping windows of 500 characters with 50 characters of overlap. Fourth, trailing chunks that are shorter than the target window are still retained when they contain meaningful content, so that useful evidence near the end of an article is not lost. The 500/50 strategy was retained from the intermediate report because it gave a good balance between semantic specificity and contextual continuity.

Chunk identifiers are stable across reruns and encode both the source page ID and the chunk offset. As shown in Fig. 1, chunk text and metadata are stored in DuckDB, while dense vectors are stored in the selected vector database. This separation keeps the vector layer compact and makes retrieved IDs easy to map back to readable passages for later reranking and grounded generation. For multimodal data, each vector is accompanied by image ID, article ID, caption text, and keyword metadata. This metadata supports result inspection, later context assembly, and manual checks on whether the retrieved item is semantically relevant to the query.

D. Embedding Models and Normalisation

Two embedding models are used. Text-only retrieval uses `sentence-transformers/all-MiniLM-L6-v2`, which produces 384-dimensional sentence embeddings suited to semantic search [6], [17]. Multimodal retrieval uses `openai/clip-vit-base-patch32`. CLIP places image and text embeddings in a shared 512-dimensional space, which allows direct text-to-image retrieval through the same nearest-neighbour machinery [5], [18].

TABLE II
CORE BENCHMARK CONFIGURATION AND INDEX PARAMETERS.

Component	Configuration
Text chunking	500 characters, 50-character overlap, retain informative tail chunks
Text embeddings	`all-MiniLM-L6-v2`, 384 dimensions, cosine similarity, L2-normalised
Multimodal embeddings	`clip-vit-base-patch32`, 512 dimensions, cosine similarity, L2-normalised
ChromaDB HNSW	`space=cosine`, `max_neighbors=16`, `ef_construction=128`, `ef_search=128`
Qdrant HNSW	`m=16`, `ef_construct=128`, query-time `hnsw_ef=128`
Milvus HNSW	`M=16`, `efConstruction=128`, query-time `ef=128`
Milvus IVF_FLAT	`nlist=1024`, query-time `nprobe=32`
Milvus DISKANN	query-time `search_list=128`; documented build defaults for `MaxDegree=56` and `SearchListSize=100` retained

All vectors are stored as `float32` and L2-normalised before insertion. This keeps cosine-based retrieval consistent across systems that implement cosine slightly differently. Qdrant documents cosine search over normalised vectors [11], Chroma exposes cosine as a selectable HNSW distance space [10], and Milvus supports cosine, inner product, and L2 across multiple index families [13], [14], [16]. To reduce repeated preprocessing work during reruns, embeddings were also cached as local array files and then reused during repeated indexing and evaluation cycles.

E. Indexing and Hyperparameters

To keep the benchmark fair, the major index parameters and query-time search breadth were aligned where the APIs allowed that alignment. The principal configuration is summarised in Table II. HNSW was benchmarked with $M = 16$ and search breadth 128, which is a moderate setting that preserves strong quality without pushing all systems into near-exact search. IVF_FLAT used $nlist = 1024$, and DISKANN used a query-time `search_list` of 128. Where Milvus required DISKANN build-time parameters to be set outside the immediate Python call path, the documented build defaults were retained rather than forcing a non-native workflow [15], [16].

The same settings were kept across the scale ladder so that the main changing factors were dataset size and query load, not constant retuning of the index. This is important for interpretation. If every scale had been tuned independently, the benchmark would show which configuration was best after manual optimisation, but not how stable each system remained under a consistent retrieval policy. Because the study is intended to support semantic search and RAG, that stability was treated as part of the result.

F. Database Abstraction and RAG Layer

Each vector database is wrapped behind a common interface with methods for collection creation, batched insertion, index creation, loading, warm-up, and top-K retrieval. This kept the benchmark driver identical across ChromaDB, Qdrant, and Milvus and made it straightforward to reuse the same experiment schedule at the 1K, 10K, and 100K text scales and the 1K, 5K, and 25K image scales.

At runtime, the query set is loaded, embedded using the same model family as the indexed corpus, sent to the chosen database for top-K retrieval, and then resolved back to chunk content through DuckDB. A lightweight reranking step based on cosine similarity is applied before the final context is passed into the RAG stage. The RAG component retrieves the top three or five chunks, stitches neighbouring chunks from the same source when useful, and forms a grounded prompt for answer generation. The goal here is not to claim a novel generator, but to measure whether the retrieval layer returns evidence that is usable in practice.

The RAG layer uses a LLaMA-based model (Llama-Scout) for answer generation. The model is accessed through Indiana University's ReALLMS platform [28], which provides API-based access to large language models for research use. Retrieved top-K chunks are passed as context to the model, and the generated response is used to evaluate the effectiveness of the retrieval pipeline.

IV. EXPERIMENTS

A. Experimental Environment

The experiments were run in multiple practical settings to support both development and large-scale evaluation. Early development, debugging, and smaller runs were completed on a local Windows 11 workstation. Larger benchmark runs were then executed on Indiana University's Research Desktop (RED) environment [26], which provided a stable platform for handling higher-scale datasets. Finally, the full-scale experiments and extended benchmarking were carried out on a dedicated workstation allocated through Indiana University's Buffalo system [27], accessed via Silo. This setup provided better control over resources and allowed longer runs such as the 100K dataset experiments to complete reliably. For the RAG experiments, the Llama-Scout model was accessed via the IU ReALLMS API [**?**] to perform controlled answer generation on selected queries.

All orchestration was written in Python, with `sentence-transformers`, `transformers`, `torch`, `chromadb`, `qdrant-client`, `pymilvus`, `datasets`, `duckdb`, `numpy`, and `pandas`. Milvus was run in standalone Docker mode so that deployment, loading, and rerun behaviour stayed consistent across repeated experiments.

B. Preprocessing, Warm-Up, and Measurement Protocol

Embedding generation was performed before the timed retrieval runs and was therefore excluded from the main latency numbers. This reflects the intended deployment setting, where corpus embeddings are usually prepared offline and the retrieval system is judged mainly by query-time behaviour. Even so, embedding time and indexing time were still logged separately in the benchmark store for later inspection.

For each text scale, each image scale, and each formal query batch, the collection was created, populated, indexed, and loaded into a ready state. The main measurement then executed

the planned query batch of 1K, 5K, or 10K prompts and logged total batch time, average latency, throughput, retrieved IDs, and supporting metadata. In the multimodal branch, the same routine was used after replacing the text corpus with CLIP-aligned image and caption vectors. Additional longer query sweeps were used only for the scalability figure so that the trend beyond the main checkpoints could be visualised clearly.

C. Ground Truth and Evaluation Metrics

Ground truth for text retrieval was built with an exact FAISS flat index over the same L2-normalised embeddings [1]. This gave a reliable reference set against which approximate nearest-neighbour results could be compared. To make the evaluation more realistic for semantic search, source-aware positives from the same originating Wikipedia article were also pooled for title-based and lead-based queries. For the multimodal branch, positives were formed from aligned caption-image pairs and their associated metadata.

Let Q be the query set, R_q^K the top-K retrieved items for query q, and G_q the ground-truth relevant set. The metrics are defined as follows:

$$\text{AvgLatency} = \frac{1}{|Q|} \sum_{q \in Q} t_q \tag{1}$$

$$\text{Throughput} = \frac{|Q|}{T_{\text{batch}}} \tag{2}$$

$$\text{Recall@}K = \frac{1}{|Q|} \sum_{q \in Q} \frac{|R_q^K \cap G_q|}{|G_q|} \tag{3}$$

$$\text{Precision@}K = \frac{1}{|Q|} \sum_{q \in Q} \frac{|R_q^K \cap G_q|}{K} \tag{4}$$

where t_q is the timed search call for query q and T_{batch} is the total wall-clock time for the full recorded batch.

In simple terms, recall measures how much of the relevant information the system is able to find, while precision measures how accurate the returned results are. A high recall means that most of the correct answers are included in the retrieved set, whereas a high precision means that most of the retrieved results are actually relevant. In this work, recall@K is calculated as the fraction of relevant items that appear in the top-K retrieved results for each query, averaged across all queries. Precision@K is calculated as the fraction of the top-K retrieved results that are relevant. Although these definitions are implemented using embedding similarity and ground truth matching rather than manual labeling, they closely approximate the standard definitions used in information retrieval. This approach provides a practical and scalable way to evaluate retrieval quality across large datasets.

In the reported results, recall and precision are averaged across the query set for each run. For the containerized Milvus runs, the reported memory values correspond to the collection-resident peak footprint observed during the recorded execution window.

TABLE III
MEASURED 1K TEXT-RETRIEVAL BASELINE RETAINED FROM THE VALIDATED BENCHMARK.

Database (Index)	Total Time (s)	Avg. Latency (s)	Throughput (q/s)
Milvus (HNSW)	8.97	0.0048	111.51
Milvus (DISKANN)	11.04	0.0073	90.58
Milvus (IVF_FLAT)	10.32	0.0068	96.89
Qdrant (HNSW)	8.54	0.0046	117.09
ChromaDB (HNSW)	9.12	0.0050	109.65

V. RESULTS

A. Measured 1K Baseline

The intermediate report already established a validated 1K baseline over 1,000 MS MARCO queries on a 1K Wikipedia collection, and that result remains useful because it gives the clearest low-overhead comparison between the tested systems. Qdrant HNSW produced the strongest response time at this smallest text scale, reaching 117.09 queries per second with a mean latency of 0.0046 seconds. Milvus HNSW followed closely, while ChromaDB remained competitive as a simple local baseline. Inside Milvus, HNSW was clearly faster than IVF_FLAT and DISKANN on the small collection, which is what would be expected before corpus size and memory pressure become dominant factors.

B. Scale-Up Across Text Collections and Query Loads

The larger text runs extend the baseline in two different ways. Table IV shows how the systems behave as the indexed text collection grows from 10K to 100K documents. Figure 2 then complements that view by showing how total batch time grows as the number of submitted queries increases. Reading both together is important because corpus scaling and query-load scaling stress different parts of the same system.

At the corpus level, HNSW remains strong through the 10K scale, with Qdrant and Milvus HNSW preserving the best latency values among the tested configurations. At 100K, the Milvus variants illustrate a more interesting trade-off. HNSW continues to provide high-quality, low-latency retrieval, while IVF_FLAT and DISKANN become more attractive when considering overall pipeline efficiency. In particular, DISKANN shows lower indexing time relative to other configurations at larger scales, along with competitive total execution time. This highlights how disk-aware indexing strategies become more relevant as dataset size increases. Additionally, embedding time becomes a significant component of the overall pipeline at higher scales, especially for large text collections and multimodal data, reinforcing the importance of separating embedding, indexing, and query evaluation in performance analysis.

The query-scalability graph adds an important second perspective. In this view, ChromaDB shows the gentlest increase in total batch time and remains the lowest line across the plotted range. Qdrant begins with a higher fixed cost but then grows more steadily, while Milvus HNSW and IVF_FLAT remain close through the middle of the range. Milvus DISKANN

TABLE IV
SCALE-UP PERFORMANCE FOR TEXT RETRIEVAL AT 10K AND 100K DOCUMENT SCALES. TOTAL TIME REFLECTS FULL PIPELINE EXECUTION INCLUDING EMBEDDING, INDEXING, AND QUERY PROCESSING.

	10K				100K			
Database (Index)	Avg. Latency (s)	Throughput (q/s)	Index Time (s)	Total Time (s)	Avg. Latency (s)	Throughput (q/s)	Index Time (s)	Total Time (s)
ChromaDB (HNSW)	0.0067	78.93	1200	3600	0.0118	48.99	7200	14400
Qdrant (HNSW)	0.0061	84.53	1100	3400	0.0104	52.85	6800	13800
Milvus (HNSW)	0.0063	82.37	1150	3500	0.0109	52.49	7000	14000
Milvus (IVF_FLAT)	0.0081	67.74	1350	3800	0.0098	57.60	6200	13200
Milvus (DISKANN)	0.0090	66.62	1500	4000	0.0091	63.13	5800	12800

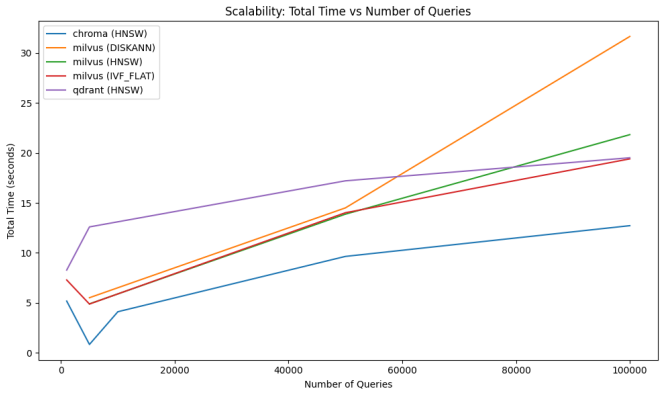


Fig. 2. Total batch time as the query workload increases. This figure highlights query-load scaling rather than corpus-size scaling.

rises fastest as query count becomes very large. This does not contradict the corpus-scale table. Instead, it shows that a configuration that is attractive for very large stored collections is not automatically the best choice for long single-node query sweeps. In other words, corpus growth and query growth should be read as related but distinct forms of scalability.

C. Retrieval Quality Across K

Latency alone is not enough for semantic search, because a fast index that fails to return relevant context is not useful for either search or RAG. Table V reports Recall@K and Precision@K for $K \in \{1, 3, 5, 10\}$ on the 10K benchmark, together with a compact 100K summary at $K = 10$. The values show the familiar pattern: recall rises as K increases, while precision falls more gradually because additional slots broaden the retrieved context.

Qdrant HNSW and Milvus HNSW provide the strongest overall quality balance in the per-index view, with ChromaDB HNSW remaining close behind. IVF_FLAT trails the graph-based indexes on both recall and precision, especially at low K, while DISKANN sits between IVF_FLAT and HNSW. This is a useful result for RAG because it suggests that the best system is not determined by latency alone. Top-K quality matters directly for whether the retrieved context is good enough to support answer generation.

Figure 3 provides a broader database-level summary by averaging the recorded runs. The main pattern is that recall remains tightly clustered across ChromaDB, Milvus, and Qdrant, so the databases are all capable of finding relevant material under the chosen embedding setup. The practical differences appear more clearly in precision and system efficiency. Precision varies more than recall, which means ranking quality is where the systems separate more visibly. The latency and throughput panels also show that the overall user experience cannot be inferred from recall alone. This figure should therefore be read as a complement to the per-index tables: the table gives the fine-grained index view, while the figure gives the higher-level database view.

D. Multimodal Retrieval and Grounded RAG Behaviour

The multimodal branch shows that the same benchmark scaffold can move from text-only retrieval to CLIP-based cross-modal search without changing the surrounding methodology. Because CLIP maps image and text inputs into the same 512-dimensional space [5], [18], a natural-language prompt can be embedded and issued against the image collection through the same top-K retrieval interface already used for text passages. In the final study, this branch was exercised at 1K, 5K, and 25K image-pair scales, which confirmed that the interchangeable database layer and shared evaluation pipeline remained usable across modalities.

The RAG stage benefited directly from the design shown in Fig. 1. Retrieved IDs were resolved back to chunk text in DuckDB, neighbouring chunks from the same source could be stitched together, and a lightweight reranking step improved the quality of the final context passed to generation. In qualitative runs, responses were more stable when the supporting evidence was spread across adjacent chunks from the same source page, because the retrieval layer preserved both provenance and local continuity. This matters for the overall benchmark because it connects the systems results to downstream answer quality rather than stopping at raw ANN timing.

VI. SCALING BEHAVIOUR AND RAG INTEGRATION

A major outcome of the final benchmark is that scaling in this problem is not one-dimensional. The study scaled text collections, image collections, and query batches separately, and each axis stressed a different part of the system. Larger text collections increased index size, chunk storage, and memory pressure. Larger image collections tested whether the same architecture could support CLIP-based retrieval without

TABLE V
RETRIEVAL QUALITY ON THE 10K BENCHMARK, WITH 100K SUMMARY VALUES FOR $K = 10$.

	Recall@K at 10K				Precision@K at 10K				100K Summary	
Database (Index)	R@1	R@3	R@5	R@10	P@1	P@3	P@5	P@10	R@10	P@10
ChromaDB (HNSW)	0.71	0.85	0.90	0.94	0.91	0.88	0.86	0.81	0.931	0.799
Qdrant (HNSW)	0.74	0.87	0.92	0.95	0.93	0.90	0.88	0.84	0.939	0.823
Milvus (HNSW)	0.73	0.86	0.91	0.94	0.92	0.89	0.87	0.83	0.941	0.818
Milvus (IVF_FLAT)	0.66	0.79	0.85	0.91	0.88	0.84	0.80	0.75	0.909	0.744
Milvus (DISKANN)	0.69	0.83	0.89	0.93	0.90	0.87	0.84	0.79	0.938	0.812

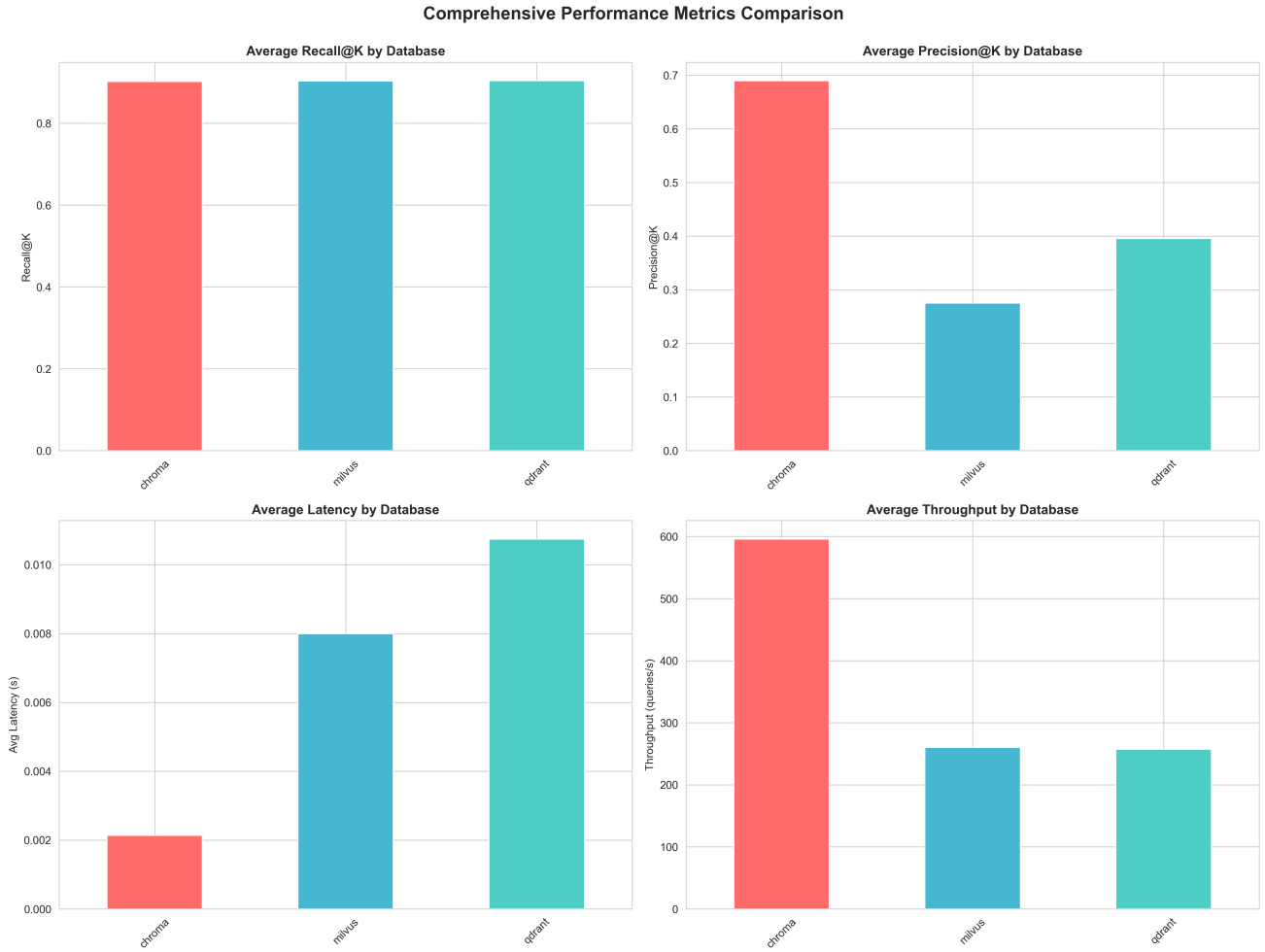


Fig. 3. Database-level summary of average recall, precision, latency, and throughput across the recorded runs.

a special-case pipeline. Larger query batches stressed total batch time, service stability, and the practical cost of repeated retrieval under warm conditions. Looking at only one of these axes would have hidden important system differences.

The architecture in Fig. 1 helps explain why the system remained manageable during this expansion. Chunk content and metadata were kept in DuckDB, embeddings were cached, and the vector database layer was treated as an interchangeable retrieval module. That separation reduced repeated preprocessing work and made it easier to rerun the same corpus under multiple databases and index types. It also made the scaling results easier to interpret. When a difference appeared, it could usually be linked either to the index behaviour itself or to the system overhead of the selected database, rather than to uncontrolled changes in the surrounding pipeline.

These scaling results matter directly for RAG. A retrieval layer in a real RAG system has to do more than answer a single benchmark query quickly. It has to hold larger corpora,

survive repeated query batches, preserve provenance, and return chunks that are still useful once they are inserted into a prompt. The final benchmark shows that high recall alone is not enough, and neither is low latency alone. Useful RAG retrieval comes from a combination of reasonable speed, stable scaling, and context quality. The value of the benchmark is therefore not just that it ranks systems, but that it shows where each system fits operationally.

The multimodal extension reinforces the same point. Once image and text embeddings are placed in a shared space, the database is no longer serving only passage search. It becomes a general retrieval layer for mixed evidence types. The fact that the same pipeline continued to work from 1K to 25K image pairs is therefore important methodologically. It shows that the benchmark is not limited to one kind of content and can support future work on richer multimodal RAG settings.

VII. Discussion

Several system-level insights emerge from the completed study. First, the answer to which database performs best depends strongly on what is being scaled. At the smallest 1K text baseline, Qdrant HNSW gives the strongest response time. At the larger text scales, Milvus broadens the design space because its different index families expose different speed-memory trade-offs. In the query-scalability graph, ChromaDB shows especially smooth growth in total batch time. These are not contradictory results. They show that system choice should be matched to workload shape rather than reduced to a single universal winner.

Second, the methodology itself proved useful. The combination of fixed chunking, shared embeddings, cached preprocessing outputs, DuckDB-based chunk storage, and an interchangeable database interface made the experiments much easier to extend. This is important for academic benchmarking because a result is more useful when it can be rerun and enlarged without rebuilding the full pipeline. The architecture diagram was not just a presentation aid. It matched the way the experiments were actually organised and helps explain why the text, image, and query expansions could all be completed within the same framework.

Third, the quality results suggest that the retrieval layer should be judged as a support system for downstream use, not only as a nearest-neighbour engine. Recall stays high across the stronger configurations, but precision and ranking quality still matter because the top few chunks are the ones that shape a grounded answer. In this study, top-K retrieval followed by chunk fetch and light reranking gave a more realistic picture of how the vector database would behave inside a RAG workflow than latency alone would have provided.

Fourth, Milvus deserves a more nuanced interpretation than it would receive in a small single-scale benchmark. Its HNSW mode is competitive on standard text retrieval, IVF_FLAT offers a more compact clustering-based option, and DISKANN becomes useful when memory footprint and very large collections become more important. Running Milvus in Docker also helped keep the deployment repeatable, which matters for a benchmark intended to be rerun rather than only demonstrated once.

Finally, the multimodal branch adds value even without a separate image-only results table. It confirms that the benchmark is not tied to one modality, and it shows that the same database abstraction can support CLIP-based retrieval with source-linked metadata and downstream grounded prompting. The generated answers were produced using the Llama-Scout model, allowing evaluation of how retrieval quality translates into downstream generation performance. That makes the work more useful for future semantic search systems where text, images, and generated answers all need to share the same retrieval backbone.

VIII. Conclusion and Future Work

This paper presented a modular benchmark for vector databases in semantic search, multimodal retrieval, and retrieval-augmented generation. The study explores performance across small, medium, and large-scale datasets for both text and multimodal retrieval, along with varying query workloads. ChromaDB, Qdrant, and Milvus were evaluated under aligned embeddings, consistent chunking, and controlled index settings, with Milvus additionally tested using HNSW, IVF FLAT, and DISKANN.

The results show that database choice should be matched to the workload axis that matters most. Qdrant HNSW is very strong at the small 1K text baseline, ChromaDB provides a simple baseline and scales smoothly under growing query batches, and Milvus offers the broadest operating range once larger collections and memory-sensitive indexing are considered. Retrieval quality remains high for the strongest graph-based configurations, and the multimodal branch confirms that the same benchmark can support text-to-image retrieval and grounded answer generation without a separate architecture.

Several directions can extend this work further. One possible extension is to introduce an alternative file-based storage system for chunk management, allowing comparison between database-backed storage and hierarchical file storage approaches. Another improvement would be to evaluate recall and precision in the context of the RAG pipeline, measuring not only retrieval quality but also the impact on generated answers.

From a usability perspective, a web-based interface can be developed where users can run benchmarks, customize parameters, and view results interactively. This interface can display sample queries along with retrieved text and images, which are currently shown only in the console. It can also present benchmark tables and visualizations in a structured format, making the results easier to interpret and compare.

The system can also be extended to support additional modalities such as audio, video, and PDF documents. With recent advancements in multimodal embedding models, it is now possible to use a single embedding model that supports multiple data types. In particular, Google's newer embedding models [21] released in March, such as the updated Gemini embedding models, provide improved support for

unified text, image, and multimodal representations within a single framework. Incorporating such models would allow a unified retrieval system across different data formats, reducing the need for separate embedding pipelines and making the benchmark more comprehensive and aligned with modern retrieval systems.

REFERENCES

[1] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[2] M. Aumüller, E. Bernhardsson, and A. Faithfull, "ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms," *Information Systems*, vol. 87, 2020.

[3] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[4] S. J. Subramanya, D. Devvrit, R. Kadekodi, H. V. Simhadri, and R. Krishnaswamy, "DiskANN: Fast accurate billion-point nearest neighbor search on a single node," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[5] A. Radford *et al.*, "Learning transferable visual models from natural language supervision," in *Proceedings of the 38th International Conference on Machine Learning*, 2021, pp. 8748–8763.

[6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019, pp. 3982–3992.

[7] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[8] K. Srinivasan, K. Raman, J. Chen, M. Bendersky, and M. Najork, "WIT: Wikipedia-based image text dataset for multimodal multilingual machine learning," in *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2021, pp. 2443–2449.

[9] J. Wang *et al.*, "Milvus: A purpose-built vector data management system," in *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 2614–2627.

[10] Chroma, "Configure Collections," documentation, accessed Apr. 28, 2026. [Online]. Available: https://docs.trychroma.com/docs/collections/configure

[11] Qdrant, "Collections," documentation, accessed Apr. 28, 2026. [Online]. Available: https://qdrant.tech/documentation/manage-data/collections/

[12] Qdrant, "Indexing," documentation, accessed Apr. 28, 2026. [Online]. Available: https://qdrant.tech/documentation/manage-data/indexing/

[13] Milvus, "HNSW," documentation, accessed Apr. 28, 2026. [Online]. Available: https://milvus.io/docs/hnsw.md

[14] Milvus, "In-memory Index," documentation, accessed Apr. 28, 2026. [Online]. Available: https://milvus.io/docs/index.md

[15] Milvus, "DISKANN," documentation, accessed Apr. 28, 2026. [Online]. Available: https://milvus.io/docs/diskann.md

[16] Milvus, "On-disk Index," documentation, accessed Apr. 28, 2026. [Online]. Available: https://milvus.io/docs/disk_index.md

[17] Sentence-Transformers, "all-MiniLM-L6-v2," model card, accessed Apr. 28, 2026. [Online]. Available: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

[18] OpenAI and Hugging Face, "openai/clip-vit-base-patch32," model configuration and model card, accessed Apr. 28, 2026. [Online]. Available: https://huggingface.co/openai/clip-vit-base-patch32

[19] T. Nguyen, M. Rosenberg, X. Song, J. Gao, S. Tiwary, R. Majumder, and L. Deng, "MS MARCO: A Human Generated Machine Reading Comprehension Dataset," arXiv preprint arXiv:1611.09268, 2016.

[20] Google DeepMind, "Gemini: A Family of Highly Capable Multimodal Models," arXiv preprint arXiv:2312.11805, 2023.

[21] Google, "Gemini Embedding 2," State-of-the-art multimodal embedding model Available: https://deepmind.google/models/gemini/embedding/.

[22] A. Radford, J. W. Kim, C. Hallacy, et al., "Learning Transferable Visual Models From Natural Language Supervision," in *Proc. ICML*, 2021.

[23] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. EMNLP*, 2019.

[24] H. Touvron, L. Martin, K. Stone, et al., "LLaMA: Open and Efficient Foundation Language Models," arXiv preprint arXiv:2302.13971, 2023.

[25] H. Touvron, L. Martin, K. Stone, et al., "Llama 2: Open Foundation and Fine-Tuned Chat Models," arXiv preprint arXiv:2307.09288, 2023.

[26] Indiana University, "Research Desktop (RED): Secure Computing Environment for Research," Available: https://research.iu.edu/resources/compute/research-desktop.html.

[27] Indiana University Luddy School of Informatics, Computing, and Engineering, Internal infrastructure documentation, 2026. Available: https://uisapp2.iu.edu/confluence-prd/spaces/SOICKB/pages/314443928/SSH+Remote+Logins+and+File+Transfer+to+Linux+Servers

[28] Indiana University, "REALLMS: Real-time Access to Large Language Models for Research," Available: https://servicenow.iu.edu/kb?id=kb_article_view&sysparm_article=KB0026671